Edge Intelligence Lab Assignment- Image capture

Adithyash BC – 25mml0002

In [13]:
```
!pip install opencv-python numpy
```

```
Requirement already satisfied: opencv-python in c:\anaconda\lib\site-packages (4.9.0.80)
Requirement already satisfied: numpy in c:\anaconda\lib\site-packages (1.21.2)
```

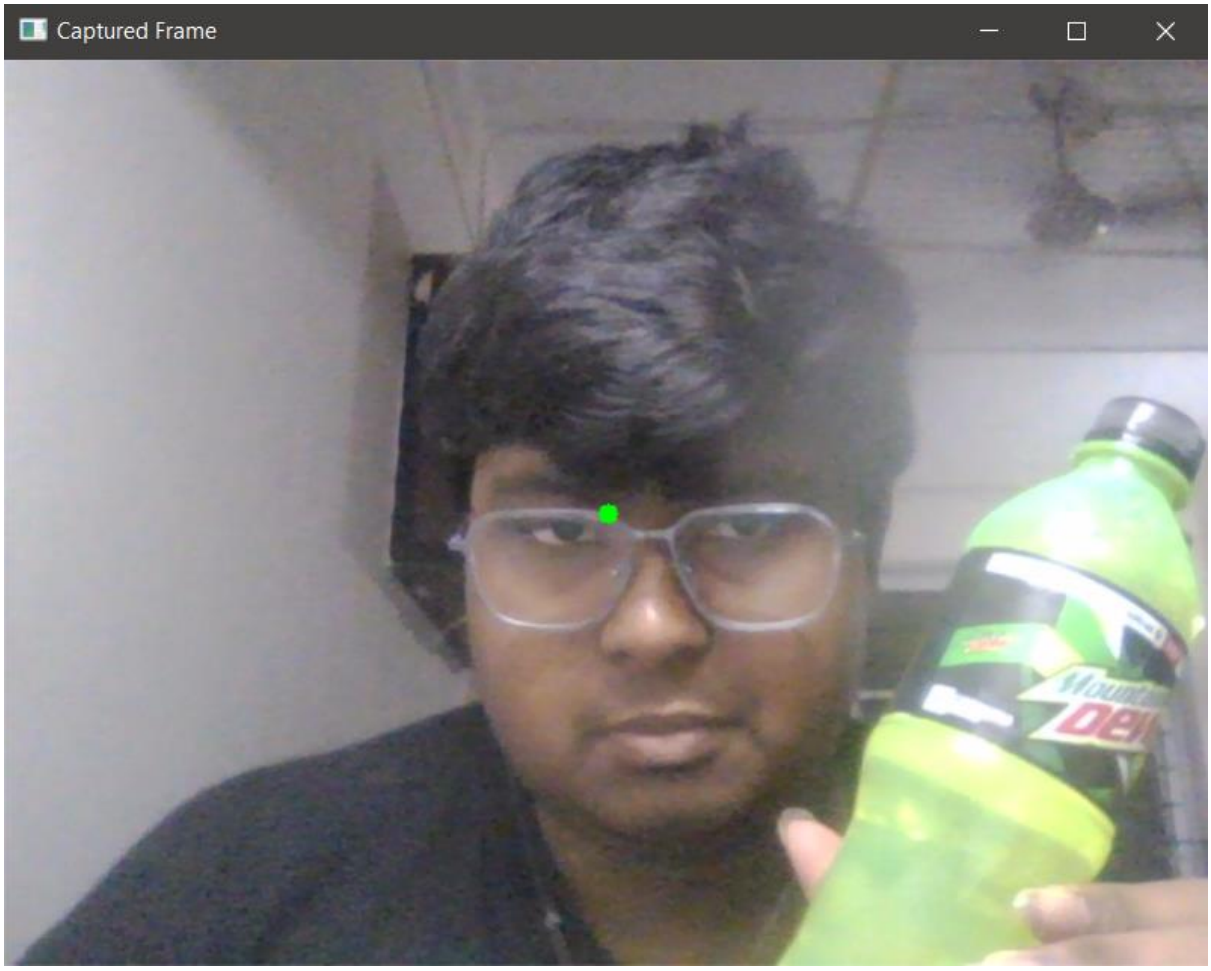
In [14]:
```
import cv2
import numpy as np
```

In [15]:
```
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Error")
else:
    print("Webcam detected successfully")

cap.release()
```

```
Webcam detected successfully
```

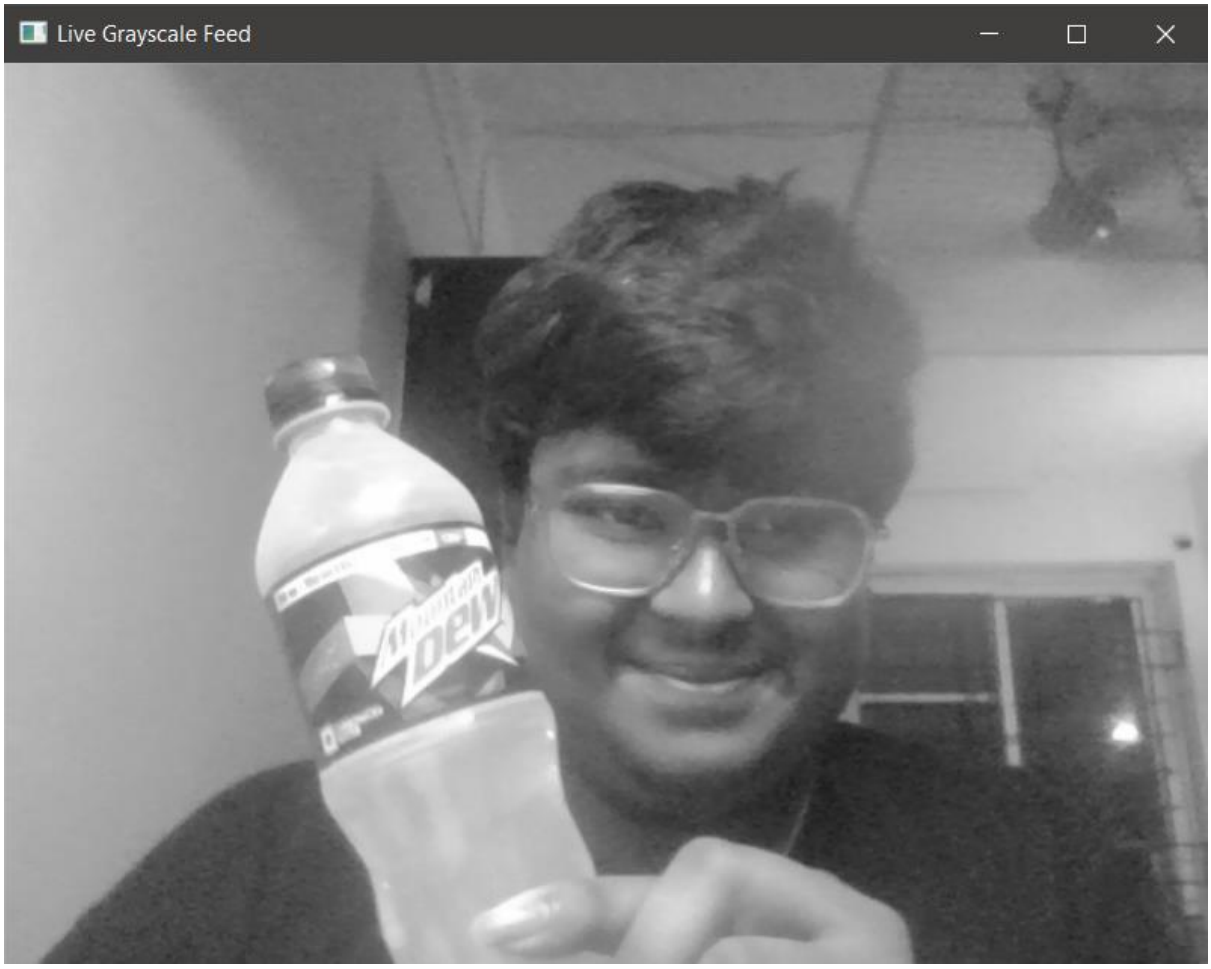
:
```
cap = cv2.VideoCapture(0)

ret, frame = cap.read()

if ret:

    h, w, _ = frame.shape

    cx = w // 2
    cy = h // 2

    B, G, R = frame[cy, cx]

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    gray_value = gray[cy, cx]

    print("Center Coordinates:", cx, cy)
    print("RGB Values:", (R, G, B))
    print("Grayscale Value:", gray_value)

    cv2.circle(frame, (cx,cy), 5, (0,255,0), -1)

    cv2.imshow("Captured Frame", frame)

    cv2.waitKey(0)

cap.release()
cv2.destroyAllWindows()
```

```
Center Coordinates: 320 240
RGB Values: (235, 254, 233)
Grayscale Value: 246
```



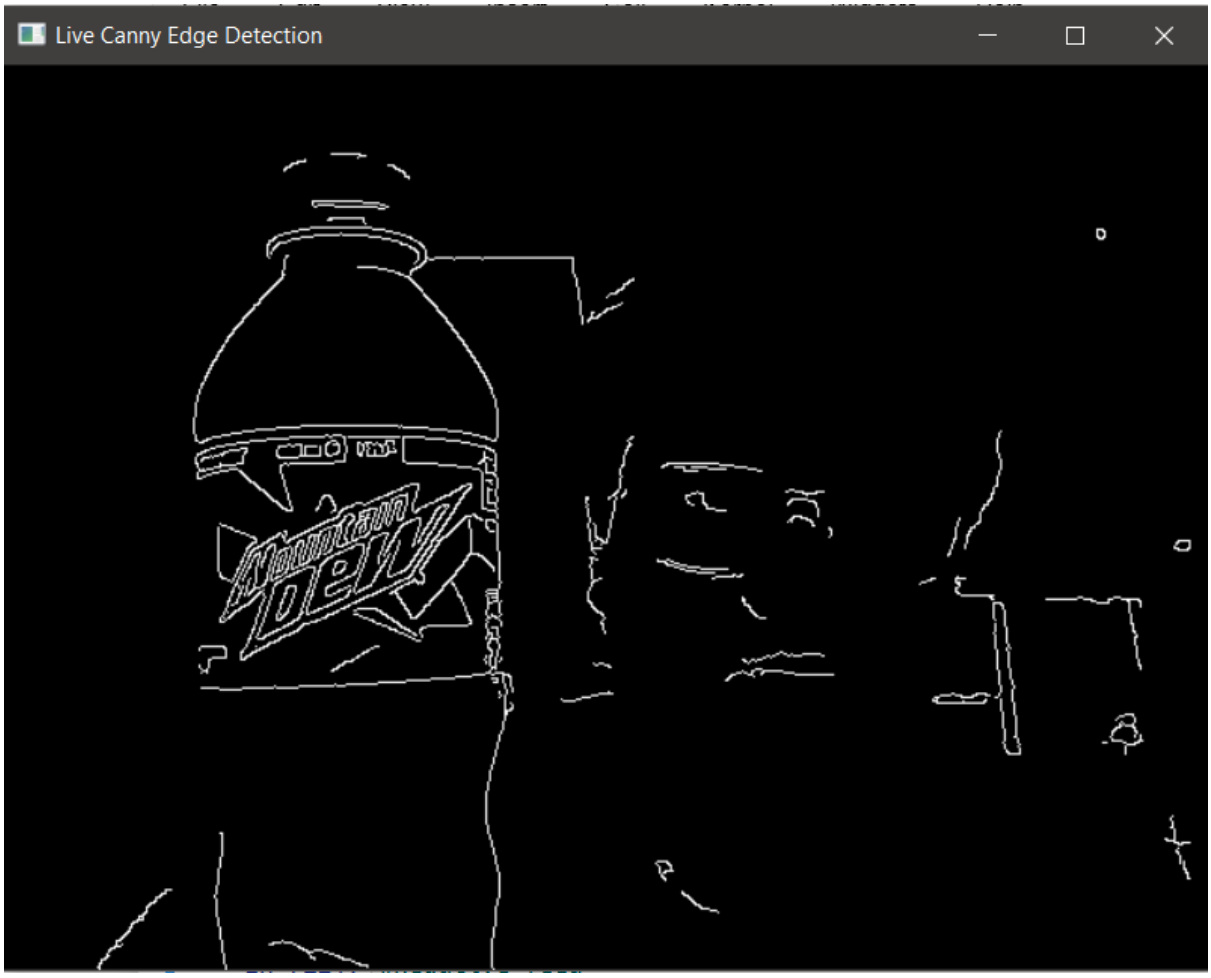

```
#grayscale feed
```

```
cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    cv2.imshow("Live Grayscale Feed", gray)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



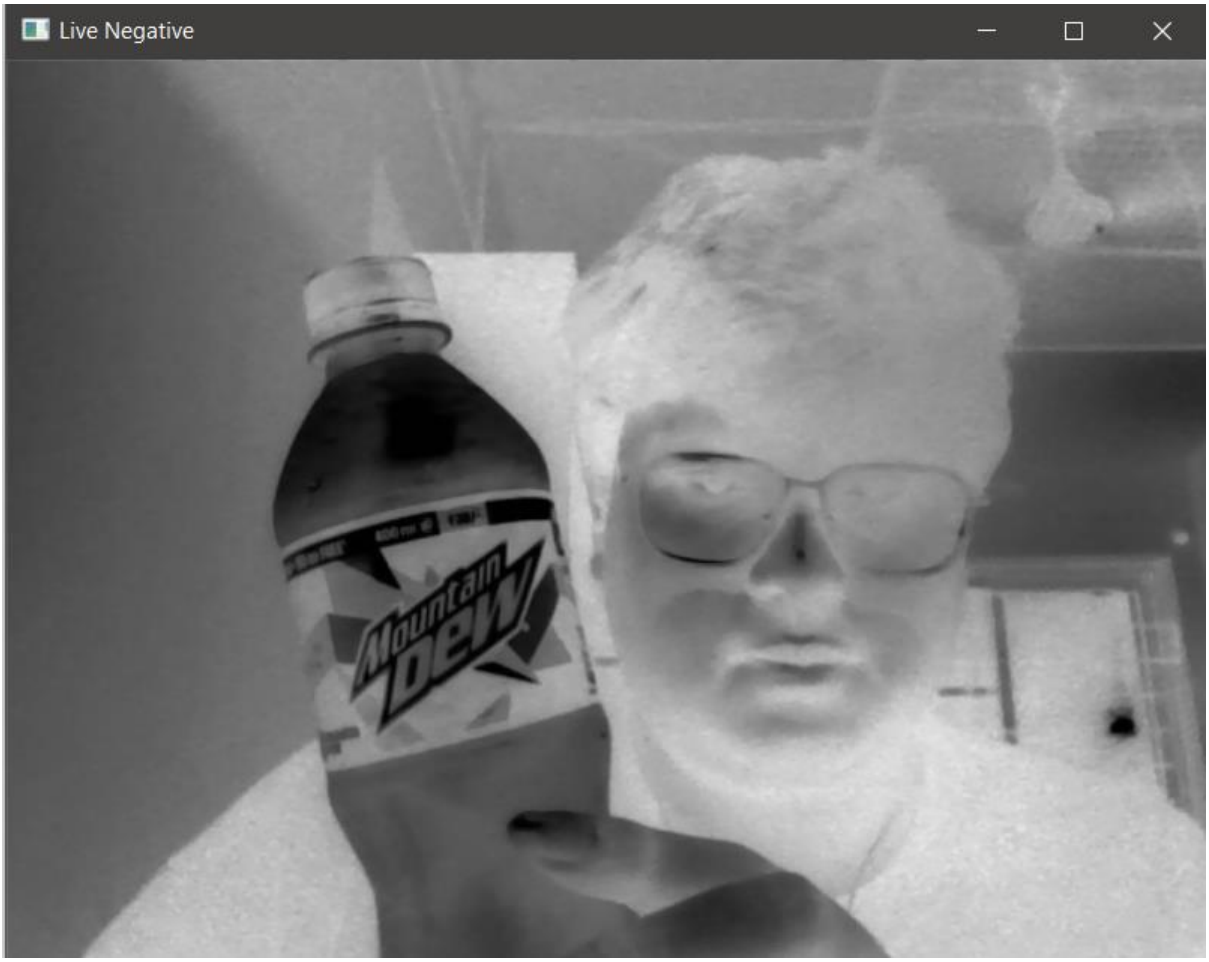

```
#canny edge detection
```

```
cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    edges = cv2.Canny(gray,100,200)

    cv2.imshow("Live Canny Edge Detection", edges)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```




```python
#negative feed
```

```python
cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    negative = 255 - gray

    cv2.imshow("Live Negative", negative)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



Comments:

I'd firstly like to preface this section with an explanation as to why I am holding a mountain dew bottle in the camera for this assignment. This assignment stated image capture, particularly frame capture from a video feed. However, while running the code, I found that canny edge detection was struggling to find the edges of my face or my spectacles. As such I devised an impromptu experiment to see if it was working as intended and since I was doing this in my hostel room I reached for the nearest piece of unambiguous object, a mountain dew bottle. The canny detection worked flawlessly in detecting the lettering of the bottle and yet my face was still an abstract mess. I changed the lighting and positions but there was a marginal difference. This led me to a few conclusions:

- The edge detection worked better on high contrast objects with well-defined boundaries
- The human face has gradual variations in colour and there are no abrupt changes to the colour or the intensity of those values

- Lighting conditions that give a defined “edge” or shadows are useful for finding out the edges of the face. Hard/harsh lighting(a cinematic term) for creating defined shadows or areas of unified contrast works well in this scenario
- My webcam is not high quality enough to capture the images in high quality(blurring/low colour gamut feed is visible in the images submitted)(not the most likely scenario but still can be contributing factor)
- My face is not defined enough(most likely)
- The parameters of canny might be tuned for high contrast images/video(also a possibility)

All of this is to ultimately justify the presence of a mountain dew bottle in the images provided.

Thank you for your patience and time.

Part 2:

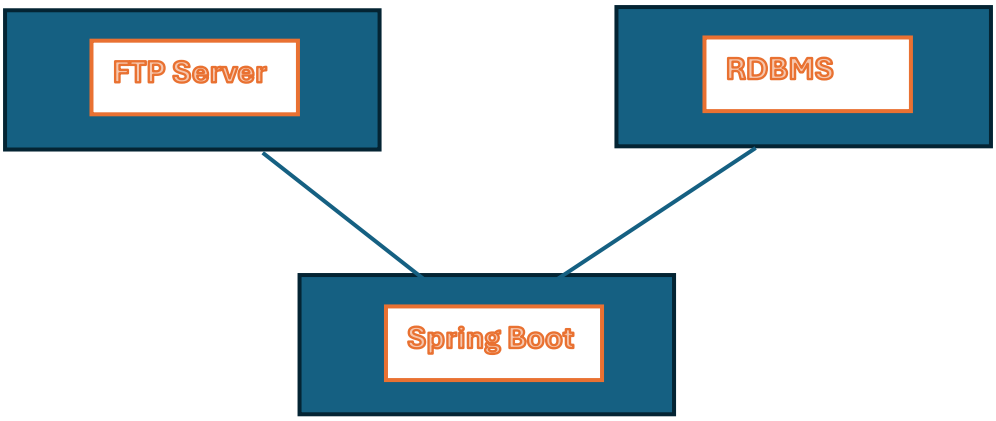
The Summary of the Presentation: Backend for the Numberplate detection using CV2- By Parth Bhatia

I would like to begin this section with a brief explanation of what exactly this backend system is supposed to do. The backend component functions primarily as a file management and processing service. It handles the uploading, storage, and retrieval of images that are later used for number plate detection. The system therefore operates more as an intermediary layer that connects the user requests, remote file storage, and the cv2 detection pipeline.

The backend is implemented using the Spring Boot framework, which provides the foundation for creating REST APIs. The application follows a layered architecture:

- The controller layer: Handles incoming HTTP requests
- The service layer: Processes business logic
- The data layer: Communicates with the database.

This separation is useful for debugging the code, as it allows each part of the application to operate independently while still contributing to the overall workflow, akin to the microservices architecture as Parth mentioned multiple times.

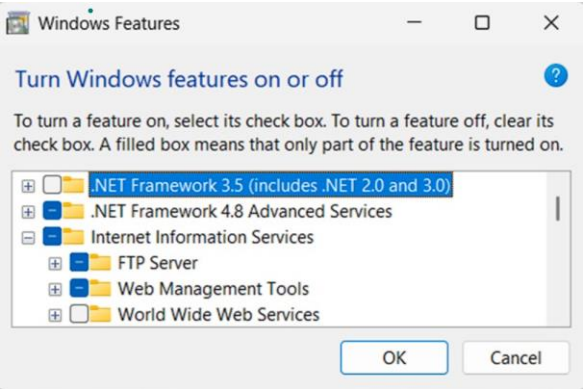


In terms of the technologies used, the system uses a combination of:

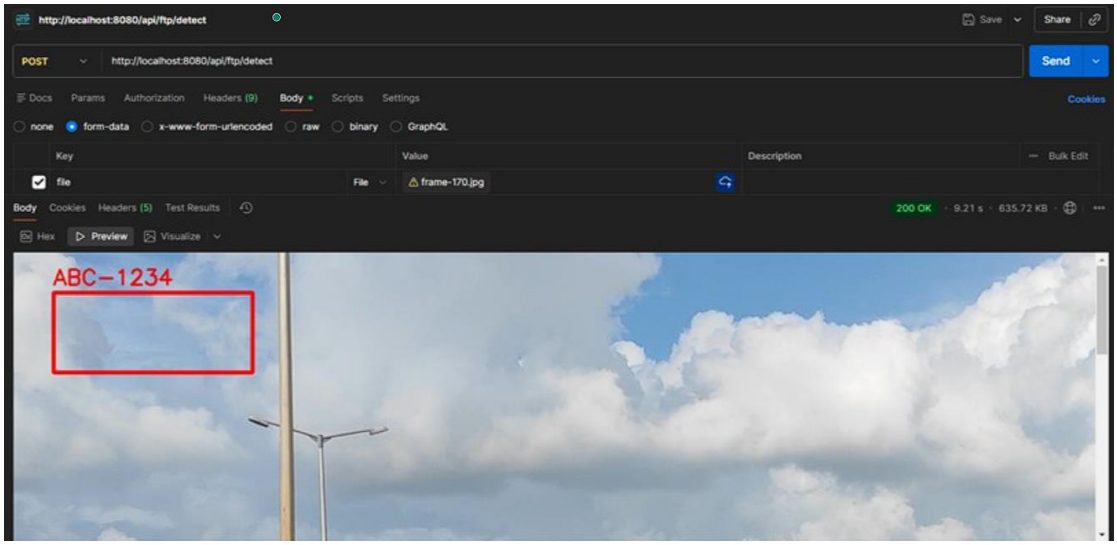
- Spring Boot
- PostgreSQL
- Hibernate
- FTP(file transfer protocol) server

The PostgreSQL database is used to maintain an audit log of uploaded files. Hibernate functions as the object relational mapping layer, allowing the application to interact with the DB via java objects rather than manually written SQL queries.

The system stores images on a remote FTP server. It uses Apache Commons Net for file operations. The FTP server was set up locally via Windows, with a dedicated site on port 21 and authentication through a Windows account.



The REST APIs allow for uploading images, retrieving them, and downloading entire directories as ZIP files. Uploaded images are processed through a machine learning model for number plate detection, and the resulting annotated images are saved alongside the originals. (AI implementation is still pending as there were some errors in the text reading and the detection, as stated by Shweta)



The images are streamed directly to the frontend. All uploads are logged in a database for Data Lineage purposes(file names, paths, timestamps)(This also helps in accountability and backtracking of data)

Query Query History

```
1 select * from file_audit_log
```

Data Output Messages Notifications

SQL Showing rows: 1 to 3

	id [PK] bigint	file_name character varying (255)	remote_path character varying (255)	upload_time timestamp without time zone (6)
1	12	frame-265.jpg	edge/numberplate/parth	2026-03-06 11:34:47.067493
2	13	frame-170.jpg	edge/numberplate/parth	2026-03-06 11:39:58.60991
3	14	frame-170.jpg	edge/numberplate/parth	2026-03-06 19:48:09.293123

Taking all of this into account, the backend is a bridge between REST APIs and a FTP storage system.

Main points of the presentation, surmised:

- Inspired by a microservices architecture model
- Uses a FTP server for storage
- Streams ZIP files rather than storing temporary files as it is a more efficient approach when handling large amounts of data
- Maintaining a database audit log; can be used for debugging or review

All of this ultimately contributes to a backend system that reliably manages storage and processing while providing the necessary APIs required for number plate detection to work.